Objects in Memory

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **describe** what the stack, heap, and static regions hold.
2. Be able to **identify** when an object becomes garbage.
3. Be able to **contrast** a shallow copy with a deep copy.
4. Be able to **use** an `ArrayList`.

Four regions of memory

Region	Holds
Code	the compiled instructions
Static memory	`static` fields – one copy per class
The stack	one frame per method call: parameters and locals
The heap	every object built with `new`

The stack is a stack of frames

Every method call pushes a **frame** holding that call's parameters and locals. When the method returns, the frame pops and every local in it is gone.

A local holding a reference goes too — but the object it pointed at does not.

→ Step through it in Java Visualizer

Where does each variable live?

The visualizer draws the stack on the left and the heap on the right. Watch `shelve()` push a frame, build an object, and pop.

→ Step through it in Java Visualizer.

The frame dies, the object does not

```
public static int booksShelved = 0;

public static Book shelve(String title) {
    Book fresh = new Book(title);
    booksShelved++;
    return fresh;
}
```

After invoking `shelve`, the frame vanishes at the closing brace. The `Book` survives because the address was returned. What if there was no `return`?

Let's talk about garbage.

An object becomes **garbage** when it is no longer reachable (simply, when it is no longer referenced). The **garbage collector** picks up on the lack of reachability and frees the memory.

```
Book mary = new Book("Project Hail Mary");
mary = null; // garbage!
```

Let's talk about garbage.

An object becomes **garbage** when it is no longer reachable (simply, when it is no longer referenced). The **garbage collector** picks up on the lack of reachability and frees the memory.

```
Book mary = new Book("Project Hail Mary");
Book phm = mary;
mary = null; // NOT garbage! Still referenced!
```

Let's talk about garbage.

An object becomes **garbage** when it is no longer reachable (simply, when it is no longer referenced). The **garbage collector** picks up on the lack of reachability and frees the memory.

```
public static void readBook() {
  Book mary = new Book("Project Hail Mary");
  mary.read();
} // after return, garbage!
```

Tophat Q1

A quick check on garbage.

→ Open Tophat to vote.

What does a copy, copy?

```
int[] a = {1, 2, 3};
int[] b = a;
int[] c = Arrays.copyOf(a, a.length); // ^
```

^ requires import of `import java.util.Arrays;`

→ Step through it in Java Visualizer

What does a copy, copy?

```
Book[] a = {new Book("Dune"), new Book("Emma")};
Book[] b = a;
Book[] c = Arrays.copyOf(a, a.length); // ^
```

^ requires import of `import java.util.Arrays;`

→ Step through it in Java Visualizer

Tophat Q2

A quick check on references and copying.

→ Open Tophat to vote.

ArrayList

The resizable list Java already wrote for you.

`ArrayList` in three lines

```
import java.util.ArrayList;

ArrayList<Book> shelf = new ArrayList<Book>();
shelf.add(new Book("Dune"));
```

It grows on demand, tracks its own size, and never leaves a hole in the middle.

You still use an **array**, not an **ArrayList**, in P2.

The four calls you need

Call	Does
`list.add(x)`	append `x` to the end
`list.get(i)`	return the element at index `i`
`list.size()`	how many elements it holds
`list.remove(i)`	delete index `i`, shift the rest down

`<TYPE>` must be a reference type

```
ArrayList<int> years;           // does NOT compile
ArrayList<Integer> years;       // this is why wrapper classes exist
years.add(1965);                // the int is autoboxed into an Integer
```

Last lecture's `Integer` was not trivia – it does get used :)

Eclipse Exercise

Download the code for today!

Callbacks

Methods you write that another program calls.

Someone else owns the loop

A GUI library runs a loop of its own, watching for **events** – a key pressed, the mouse moved, a frame due to be drawn.

When one happens it reaches into *your* class and calls a method by an agreed-on name and signature. It **must** match what was given to you in the starter code!

```
public static void keyPressed(char key) { /* your code */ }
```

You never write a call to this. The library does.

Questions?